[Министерство образования …]
[Полное наименование образовательной организации]

Специальность 09.02.07 «Информационные системы и программирование»

ОТЧЁТ

по [учебной / производственной] практике
ПМ.[__] «[наименование профессионального модуля]»

Тема: «Разработка веб-приложения "Ремонтная мастерская"»

Выполнил студент группы 9ИС-360

Панов Д. С. _____

Руководитель практики

[Фамилия И. О.] _____

Оценка: _____

Сроки практики: [__.__.2026 – __.__.2026]

[Город] 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....4
1 Анализ предметной области.....6
1.1 Описание предметной области.....6
1.2 Обзор аналогов.....6
1.3 Постановка задачи.....7
2 Проектирование.....8
2.1 Функциональные требования.....8
2.2 Нефункциональные требования.....8
2.3 Архитектура приложения.....9
2.4 Проектирование базы данных.....11
2.5 Проектирование интерфейса.....13
3 Реализация.....14
3.1 Средства разработки.....14
3.2 Структура проекта.....14
3.3 Модуль пользователей и разграничение доступа.....14
3.4 Каталог услуг и создание заявки.....15
3.5 Личный кабинет клиента.....16
3.6 Рабочее место мастера.....17
3.7 Учёт запчастей.....18
3.8 Панель администратора.....19
3.9 Статистика.....20
3.10 Адаптивность и безопасность.....21
4 Тестирование.....24
4.1 Автоматические тесты.....24
4.2 Проверка форм и граничных значений.....24
4.3 Время отклика.....25
4.4 Ручное тестирование и найденные дефекты.....25
5 Развёртывание и публикация.....27
ЗАКЛЮЧЕНИЕ.....29

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....30
Приложение А Фрагменты исходного кода.....31
Приложение Б Учётные записи и ссылки.....33

ВВЕДЕНИЕ

Сервисные центры и небольшие мастерские по ремонту электроники ежедневно принимают десятки устройств. Без информационной системы заявки ведутся в тетради или электронной таблице: клиенту приходится звонить, чтобы узнать статус ремонта, история обращений теряется, а расход запчастей на складе трудно сопоставить с конкретными ремонтами. Поэтому разработка веб-приложения, в котором клиент сам оформляет заявку и следит за ремонтом, а сотрудники ведут работу и склад в одной системе, является актуальной задачей.

Тема практики – разработка веб-приложения «Ремонтная мастерская»: приём заявок на ремонт техники, отслеживание статуса, учёт запчастей.

Цель практики – закрепление теоретических знаний и получение практических навыков проектирования и разработки программного продукта: от анализа требований до тестирования и публикации, с использованием современных языков программирования и баз данных.

Для достижения цели были поставлены следующие **задачи**:

1) провести анализ предметной области (сервисные центры, мастерские по ремонту электроники) и существующих аналогов;
2) сформулировать функциональные и нефункциональные требования;
3) спроектировать архитектуру приложения и пользовательский интерфейс;
4) спроектировать базу данных: ER-диаграмму, таблицы, связи, ограничения целостности;
5) реализовать адаптивную клиентскую часть и функциональные модули: каталог услуг, регистрацию и авторизацию с разграничением ролей, создание заявки с фото, личный кабинет клиента, рабочее место мастера, учёт запчастей, панель администратора, страницу статистики;
6) провести функциональное тестирование, проверку форм и граничных значений;

7) подготовить техническую документацию и руководство пользователя;
8) опубликовать проект, подготовить отчёт.

Объект разработки – процесс приёма и выполнения заявок на ремонт в мастерской. **Предмет разработки** – веб-приложение, автоматизирующее этот процесс для трёх ролей: клиента, мастера и администратора.

В качестве средств разработки выбраны язык Python и веб-фреймворк Django, система управления базами данных PostgreSQL, для клиентской части – HTML, CSS и JavaScript, для развёртывания – Docker. Исходный код хранится в системе контроля версий Git.

1 Анализ предметной области

1.1 Описание предметной области

Мастерская по ремонту техники принимает от клиентов неисправные устройства – смартфоны, ноутбуки, планшеты, игровые консоли, – проводит диагностику, согласует стоимость, выполняет ремонт и выдаёт устройство обратно. Типовой жизненный цикл заявки: приём, диагностика, ремонт, готовность к выдаче, выдача.

В процессе участвуют три роли. Клиент сдаёт устройство, описывает неисправность, узнаёт статус и забирает устройство. Мастер диагностирует и ремонтирует, списывает запчасти, оставляет комментарии. Администратор (приёмщик или руководитель) распределяет заявки между мастерами, ведёт прайс-лист и склад, контролирует показатели работы.

Без информационной системы возникают типичные проблемы:

– клиенты звонят, чтобы узнать статус, и отвлекают мастеров;
– история ремонтов устройства теряется в записях;
– непонятно, какие запчасти ушли в какой ремонт, из-за чего на складе возникают расхождения;
– нет данных для решений: какие поломки встречаются чаще, кто из мастеров перегружен, какой доход принёс месяц.

1.2 Обзор аналогов

Для автоматизации сервисных центров существуют готовые решения. Их сравнение приведено в таблице 1.

Таблица 1 – Сравнение существующих решений

Решение	Тип	Достоинства	Ограничения для небольшой мастерской
RemOnline	Облачная CRM для сервисных центров	Заказы, склад, касса, уведомления	Платная подписка, избыточна для 2–3 мастеров
LiveSklad	Облачная CRM для сервисов	Заказы, склад, статусы для клиента	Платная, интерфейс ориентирован на сотрудников

Решение	Тип	Достоинства	Ограничения для небольшой мастерской
Конфигурации 1С для сервисных центров	Настольное ПО	Интеграция с бухгалтерией	Дорогое внедрение, нет личного кабинета клиента
Электронные таблицы	Универсальный инструмент	Бесплатно и просто	Нет ролей, прав и истории, клиент не видит статус

Вывод: для небольшой мастерской востребовано лёгкое веб-приложение, где клиент сам оформляет заявку и следит за статусом, а сотрудники ведут ремонт и склад в одной системе без платной подписки.

1.3 Постановка задачи

Требуется разработать веб-приложение со следующими возможностями: публичный каталог услуг с ориентировочными ценами; регистрация и вход с ролями «Клиент», «Мастер», «Администратор»; создание заявки на ремонт с типом устройства, описанием неисправности и фотографиями; отслеживание статуса и история обращений; рабочее место мастера; учёт запчастей на складе; панель администратора; страница статистики с графиками.

В рамки проекта не входят онлайн-оплата, касса и фискальные чеки, SMS-уведомления и интеграция с поставщиками запчастей.

2 Проектирование

2.1 Функциональные требования

Функциональные требования сгруппированы по ролям пользователей и приведены в таблице 2.

Таблица 2 – Функциональные требования

Роль	Требования
Клиент	Просмотр каталога услуг и цен, поиск и фильтр Регистрация и авторизация Создание заявки на ремонт с прикреплением до 5 фото Отслеживание статуса: принята, в диагностике, в ремонте, готова к выдаче, выдана Просмотр истории обращений, переписка с мастером
Мастер	Просмотр назначенных заявок Изменение статуса и добавление комментариев, в том числе внутренних Указание использованных запчастей и стоимости работ Просмотр остатков склада
Администратор	Назначение заявок мастерам, поиск и фильтрация заявок Управление услугами, типами устройств, сотрудниками и клиентами Управление складом: приход, инвентаризация, журнал движений Просмотр статистики и графиков

2.2 Нефункциональные требования

Таблица 3 – Нефункциональные требования и способы их обеспечения

Требование	Как обеспечено
Адаптивный и понятный интерфейс	Вёрстка «от мобильного», сворачиваемое меню на ширине менее 860 px, тёмная тема по настройке системы
Хеширование паролей	Алгоритм PBKDF2-SHA256 с солью, проверка сложности пароля
Защита от SQL-инъекций	Доступ к данным только через ORM с параметризованными запросами
Валидация вводимых данных	Серверная проверка во всех формах, ограничения CHECK и UNIQUE в базе данных
Время отклика не более 2–3 с	Индексы, выборка связанных объектов одним запросом, постраничный вывод
Разграничение доступа	Проверка роли и владельца заявки на каждом запросе
Защита от CSRF и XSS	CSRF-токены во всех формах, автоматическое экранирование в шаблонах

2.3 Архитектура приложения

Приложение построено по трёхуровневой схеме «клиент – сервер приложений – база данных» с серверным формированием страниц. Браузер обращается по HTTPS к веб-серверу Apache, который передаёт запросы серверу приложений Gunicorn. Внутри Django запрос проходит маршрутизацию (urls.py), обработчик (views.py) проверяет роль пользователя, формы (forms.py) проверяют данные, а операции, затрагивающие несколько таблиц, выполняет слой бизнес-логики (services.py) в транзакции. Схема приведена на рисунке 1.

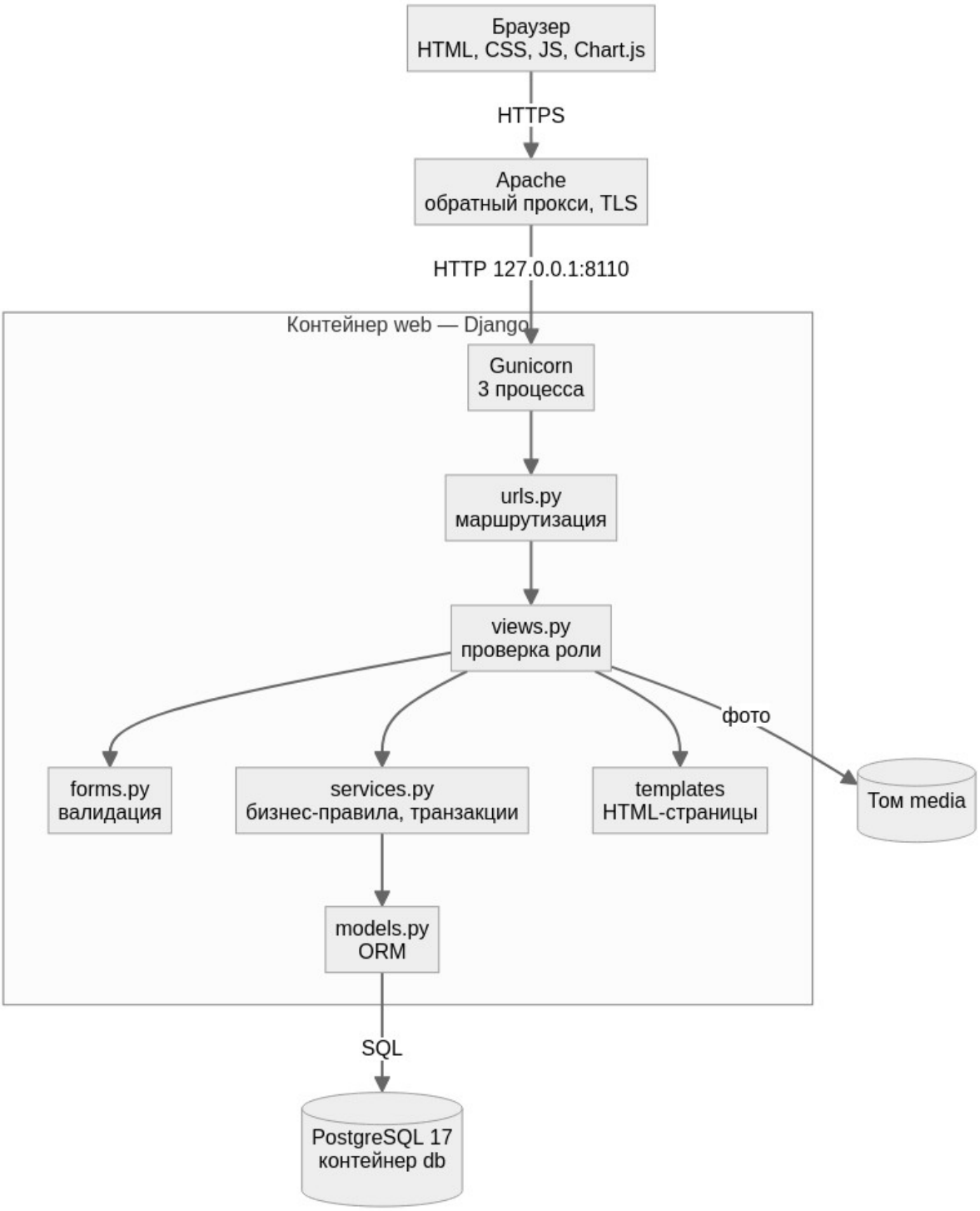


Рисунок 1 – Архитектура приложения

Код разделён на пять модулей (приложений Django): accounts – пользователи и роли, catalog – типы устройств и услуги, repairs – заявки, статусы, комментарии, фото и использованные запчасти, inventory – склад, dashboard – панель администратора и статистика.

Статус заявки меняется по фиксированной схеме переходов (рисунок 2). Мастер может перевести заявку только на соседний допустимый этап; шаг назад разрешён, чтобы исправить ошибку. Администратор может установить любой статус, например вернуть выданное устройство в ремонт по гарантии. Выдать заявку без назначенного мастера нельзя, после выдачи запчасти и стоимость работ изменить нельзя.

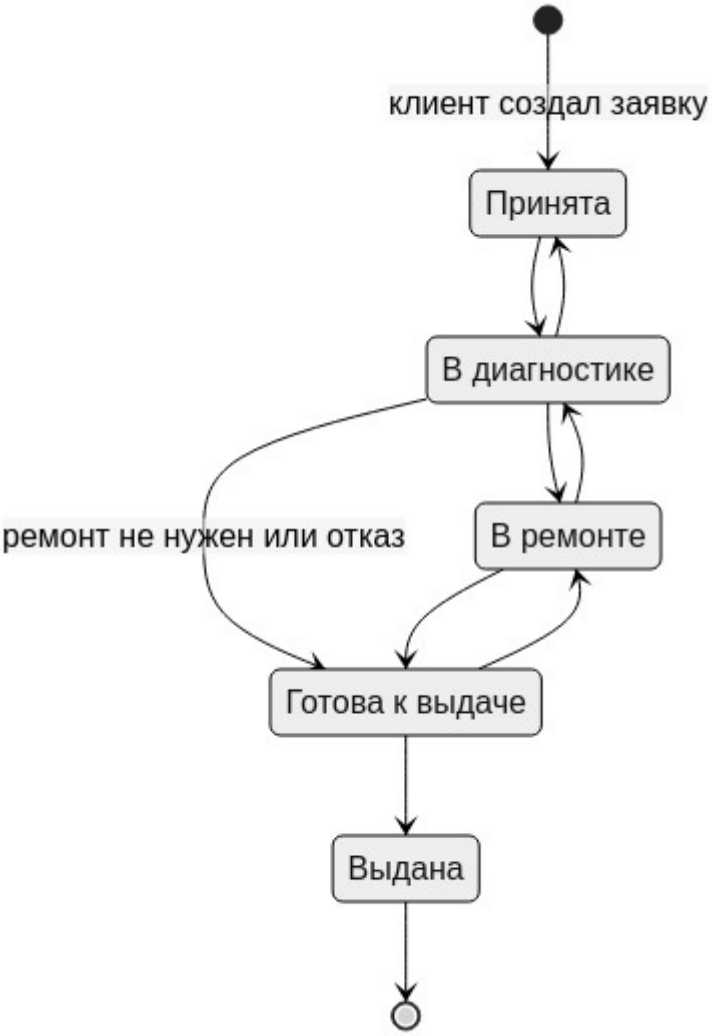


Рисунок 2 – Схема переходов статусов заявки

2.4 Проектирование базы данных

База данных содержит 11 таблиц. ER-диаграмма приведена на рисунке 3, назначение таблиц – в таблице 4.

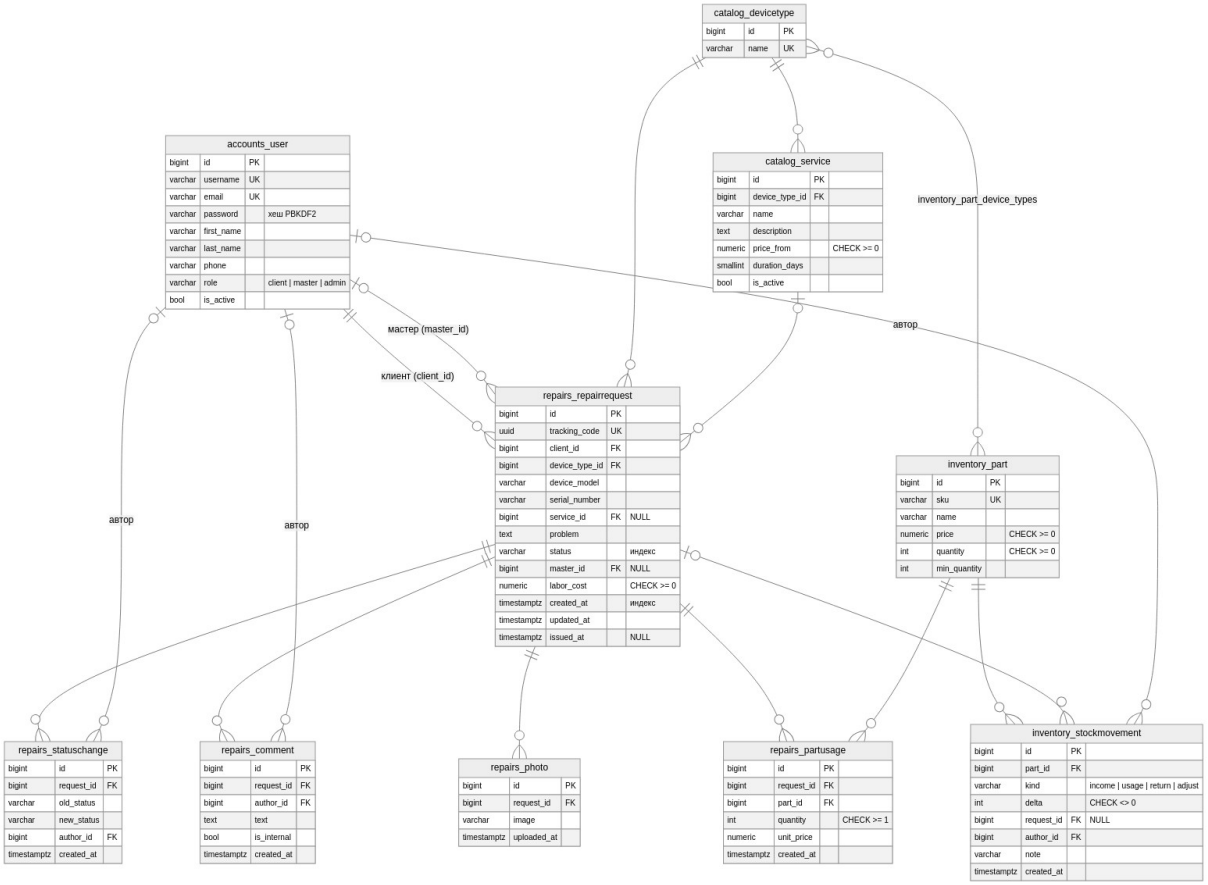


Рисунок 3 – ER-диаграмма базы данных

Таблица 4 – Таблицы базы данных

Таблица	Назначение
accounts_user	Пользователи всех ролей: логин, e-mail (уникальный), хеш пароля, ФИО, телефон, роль
catalog_devicetype	Справочник типов устройств
catalog_service	Услуги прайс-листа с ценой «от» и сроком выполнения
repairs_repairrequest	Заявки на ремонт – центральная таблица
repairs_statuschange	Журнал смены статусов: кто и когда
repairs_comment	Комментарии к заявке, признак «только для сотрудников»
repairs_photo	Фото устройства (путь к файлу)
repairs_partusage	Запчасти, использованные в ремонте, с ценой на момент списания
inventory_part	Запчасти на складе: артикул, цена, остаток, минимальный остаток
inventory_part_device_types	Связь «многие ко многим»: к каким устройствам подходит запчасть
inventory_stockmovement	Журнал движений склада: приход, списание, возврат, корректировка

Целостность данных обеспечивается ограничениями на уровне СУБД (таблица 5). Для внешних ключей выбрано поведение при удалении: PROTECT – нельзя удалить клиента, тип устройства или запчасть, на которые ссылается история; SET NULL – при удалении сотрудника история его действий сохраняется; CASCADE – фото, комментарии и история удаляются вместе с заявкой.

Таблица 5 – Ограничения целостности

Ограничение	Таблица	Смысл
UNIQUE(username), UNIQUE(email)	accounts_user	Один аккаунт на логин и почту
UNIQUE(device_type_id, name)	catalog_service	Нет дублей услуги для одного типа устройства
CHECK(price_from >= 0)	catalog_service	Цена не отрицательная
CHECK(labor_cost >= 0)	repairs_repairrequest	Стоимость работ не отрицательная
UNIQUE(tracking_code)	repairs_repairrequest	Уникальный код для проверки статуса
CHECK(quantity >= 1)	repairs_partusage	Списывается хотя бы одна штука
UNIQUE(sku), CHECK(quantity >= 0)	inventory_part	Уникальный артикул, остаток не уходит в минус
CHECK(delta <> 0)	inventory_stockmovement	Нет пустых движений

Для ускорения списков и статистики созданы индексы по полям status и created_at таблицы заявок, а также индексы по всем внешним ключам.

2.5 Проектирование интерфейса

Интерфейс спроектирован для трёх ролей. Верхнее меню меняется в зависимости от роли: клиент видит каталог, свои заявки и кнопку «Сдать в ремонт», мастер – свои ремонты и склад, администратор – заявки, склад, услуги, сотрудников, клиентов и статистику. Карточка заявки общая для всех ролей, но набор действий в ней зависит от прав пользователя. Цветовая схема нейтральная с одним акцентным цветом; статусы обозначены цветными метками, этапы ремонта – шкалой прогресса.

Макеты интерфейса в Figma: [ссылка на макет Figma].

3 Реализация

3.1 Средства разработки

Таблица 6 – Используемые технологии

Слой	Технология	Обоснование выбора
Серверная часть	Python 3.13, Django 6.0	Входит в рекомендованный стек; встроенные ORM, миграции, авторизация, формы с валидацией, защита от CSRF
Клиентская часть	HTML5, CSS3, JavaScript	Серверный рендеринг шаблонов без отдельного SPA
Графики	Chart.js 4	Столбчатые, линейные и круговые диаграммы
База данных	PostgreSQL 17, SQLite	PostgreSQL на сервере, SQLite для быстрого старта и тестов
Развёртывание	Docker Compose, Gunicorn, Apache	Одинаковое окружение локально и на сервере
Инструменты	Git, GitHub, VS Code, Firefox	Контроль версий, разработка, проверка интерфейса

3.2 Структура проекта

Проект имеет следующую структуру каталогов:

```
config/        настройки проекта и корневые маршруты
apps/accounts/ пользователь с ролью, регистрация, вход, проверка ролей
apps/catalog/  типы устройств и услуги, публичный каталог
apps/repairs/  заявки, статусы, комментарии, фото, запчасти в ремонте
apps/inventory/ склад: запчасти и журнал движений
apps/dashboard/ панель администратора и статистика
templates/     HTML-шаблоны страниц
static/        стили и скрипты
docs/          документация
deploy/        скрипты развёртывания и резервного копирования
```

Объём исходного кода – около 1 900 строк на Python, около 740 строк автотестов и около 1 050 строк шаблонов, стилей и скриптов.

3.3 Модуль пользователей и разграничение доступа

Стандартная модель пользователя Django расширена полями «роль» и «телефон», e-mail сделан уникальным. При самостоятельной регистрации роль всегда «Клиент»: даже если злоумышленник добавит в запрос поле role, оно будет проигнорировано. Сотрудников создаёт администратор.

Доступ к страницам проверяют декораторы client_required, staff_required и admin_required. Доступ к конкретной заявке проверяют функции can_view и can_work_on: клиент видит только свои заявки, мастер – только назначенные ему. На чужую заявку возвращается ответ 404, чтобы не раскрывать, что заявка с таким номером существует.

3.4 Каталог услуг и создание заявки

Главная страница – каталог услуг, сгруппированных по типам устройств, с поиском и фильтром (рисунок 4). Кнопка «Заказать» открывает форму заявки с уже выбранной услугой.

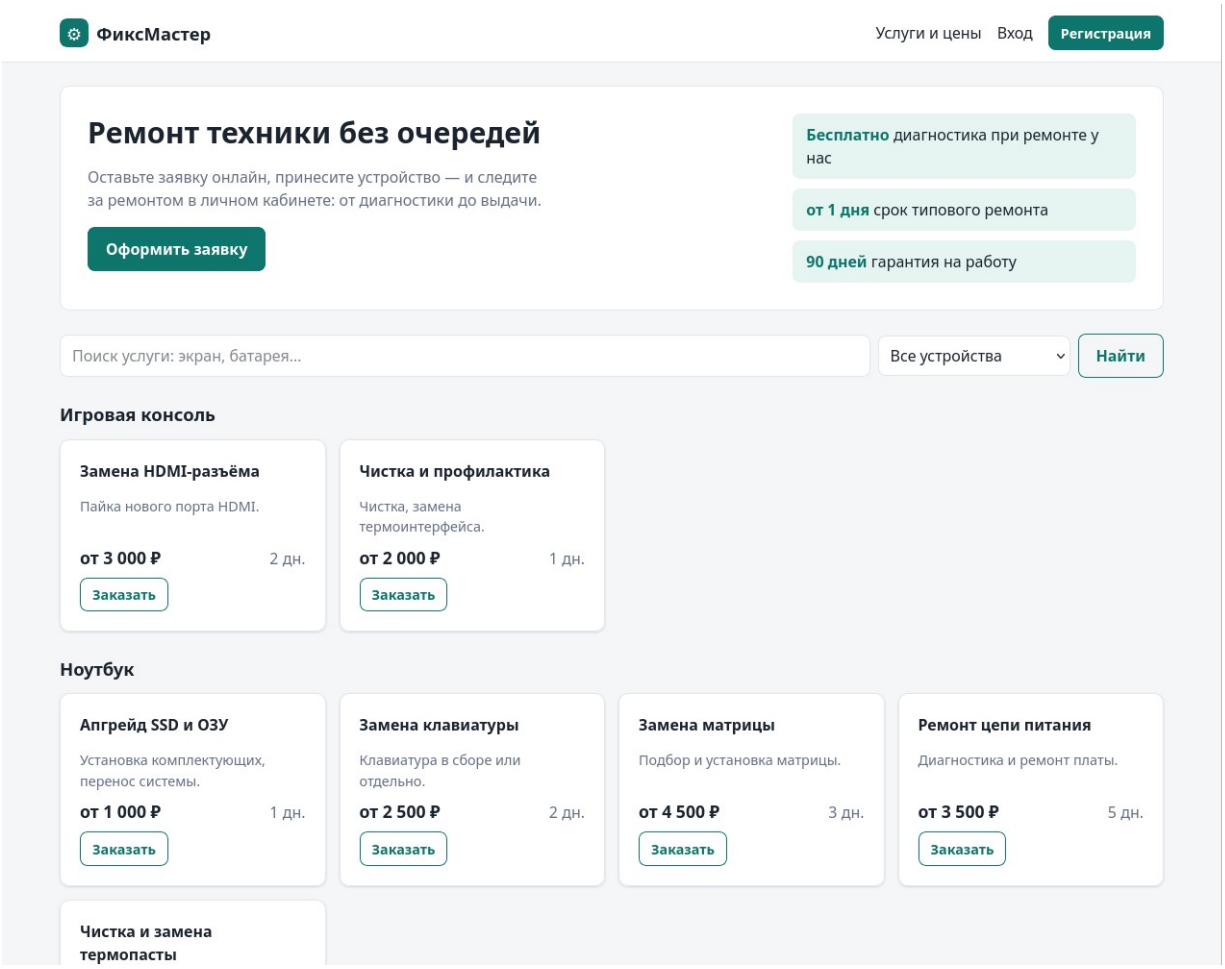


Рисунок 4 – Каталог услуг

Форма заявки (рисунок 5) проверяет: длину описания (от 10 до 2000 символов), соответствие услуги типу устройства, количество фото (не более 5), размер каждого файла (не более 5 МБ) и то, что файл действительно

является изображением. Проверка формата выполняется библиотекой Pillow, поэтому переименованный исполняемый файл не пройдёт.

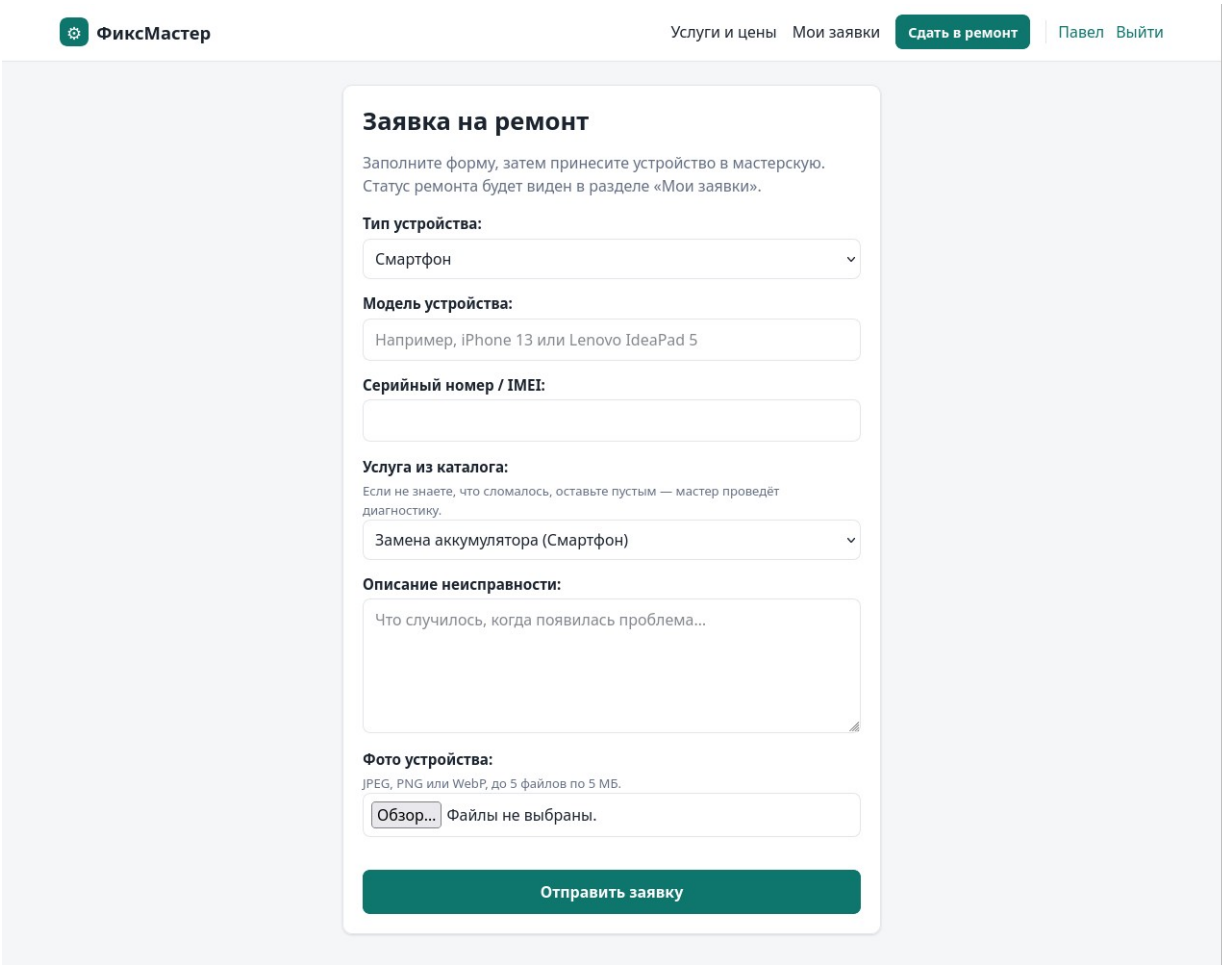


Рисунок 5 – Форма создания заявки

3.5 Личный кабинет клиента

В разделе «Мои заявки» клиент видит активные заявки со статусами и историю выданных с итоговой суммой. Карточка заявки (рисунок 6) показывает шкалу этапов ремонта, мастера, стоимость (работа и запчасти), историю статусов и переписку с мастером. Внутренние комментарии сотрудников клиенту не показываются.

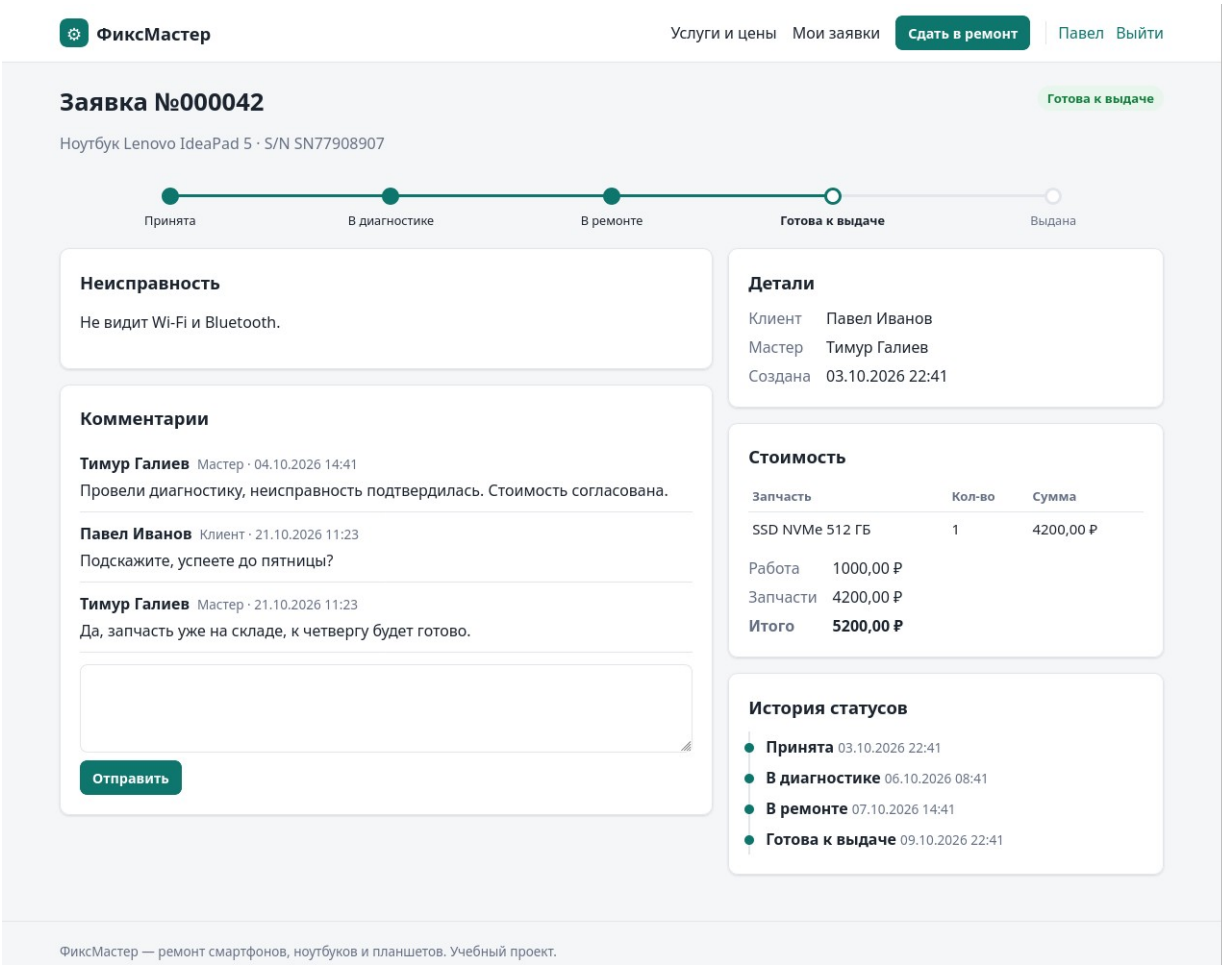


Рисунок 6 – Карточка заявки глазами клиента

Для каждой заявки генерируется случайный код отслеживания (UUID). По ссылке с этим кодом статус можно проверить без входа в систему; страница показывает только номер, устройство и этап, без личных данных.

3.6 Рабочее место мастера

Мастер видит список назначенных ему заявок с фильтром по статусу. В карточке заявки (рисунок 7) он меняет статус (в списке только допустимые переходы), пишет комментарии, указывает стоимость работ и списывает запчасти со склада. В списке запчастей сначала показываются подходящие к типу устройства.

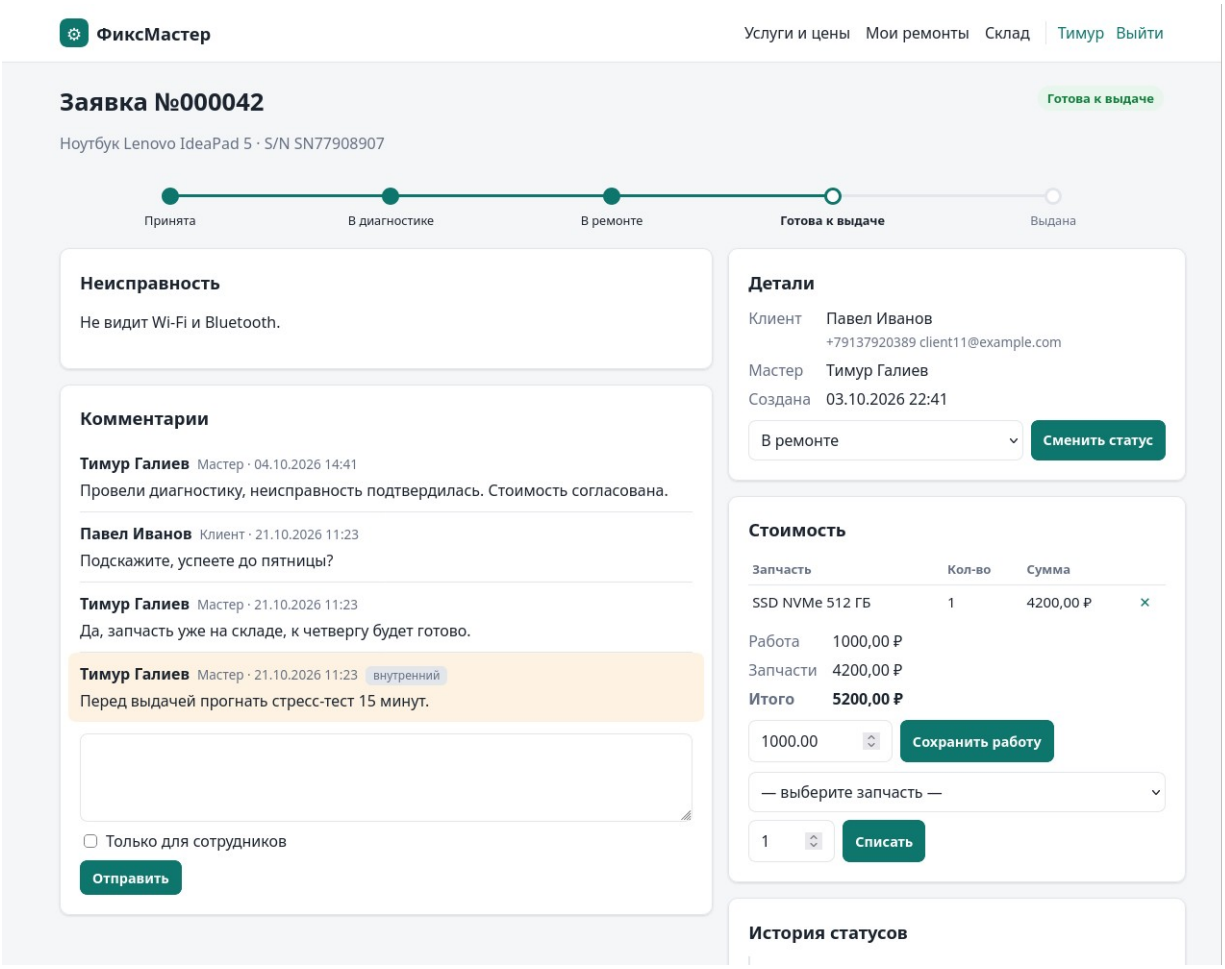


Рисунок 7 – Карточка заявки глазами мастера

Списание запчасти выполняется функцией use_part в одной транзакции: уменьшается остаток, создаётся запись в журнале склада и строка использованной запчасти с ценой на момент списания. Строка запчасти блокируется запросом SELECT … FOR UPDATE, поэтому два мастера не смогут одновременно взять последнюю деталь. Фрагмент кода приведён в приложении А.

3.7 Учёт запчастей

Склад (рисунок 8) содержит карточки запчастей с артикулом, ценой, остатком и минимальным остатком. Запчасти с остатком не выше минимального подсвечиваются, их можно отфильтровать. Остаток меняется только через движения: приход, списание в ремонт, возврат и инвентаризацию с указанием причины. Поэтому сумма движений всегда объясняет текущий остаток. Мастер видит склад, но изменять его может только администратор.

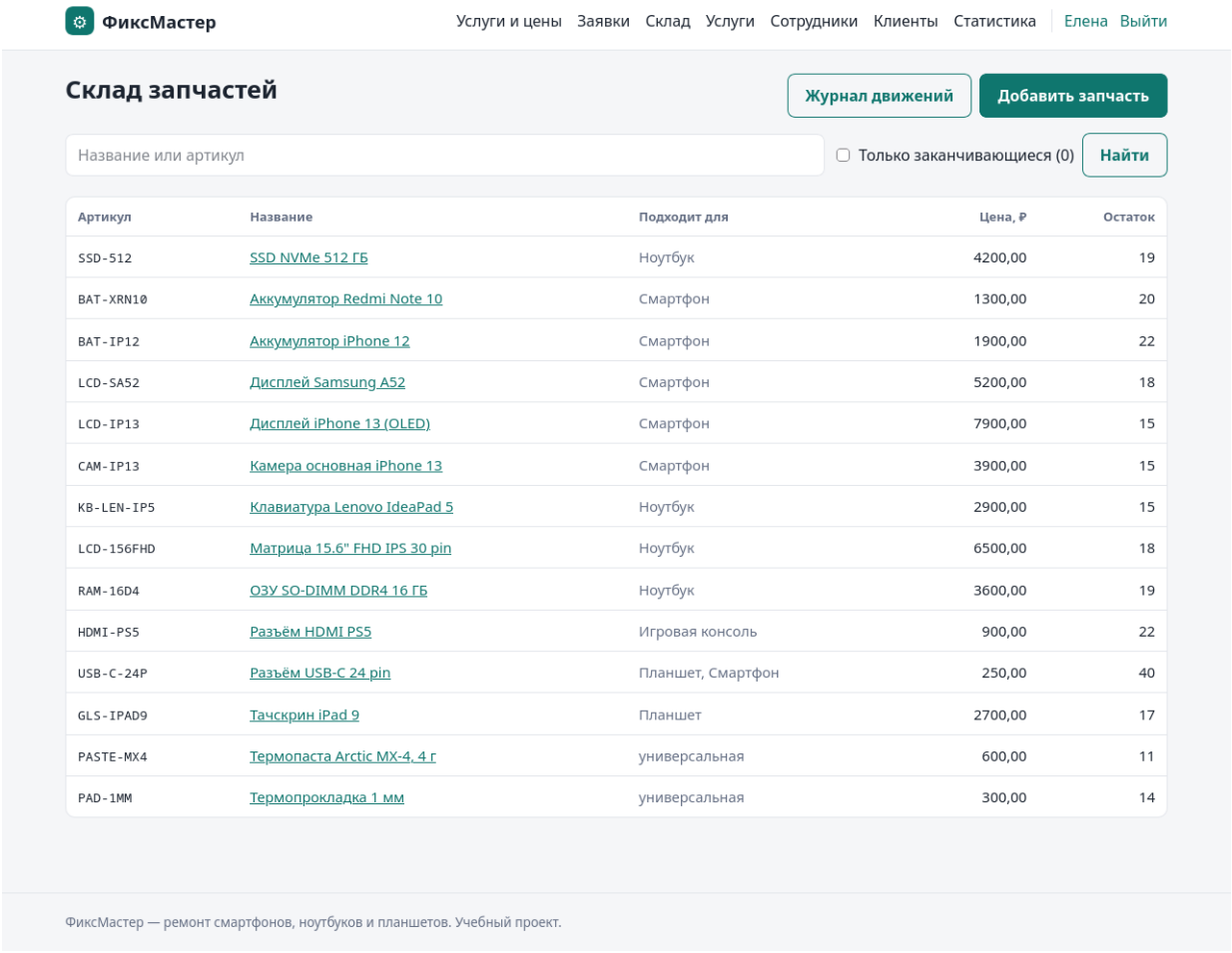


Рисунок 8 – Склад запчастей

3.8 Панель администратора

Администратор работает со списком всех заявок (рисунок 9): поиск по номеру, модели, серийному номеру, клиенту и телефону, фильтры по статусу и мастеру, отдельный фильтр «не назначен». Мастер назначается в карточке заявки. Также администратор управляет услугами и типами устройств, сотрудниками (создание, смена роли, блокировка) и клиентами. Защита от ошибки: администратор не может заблокировать или понизить собственную учётную запись.

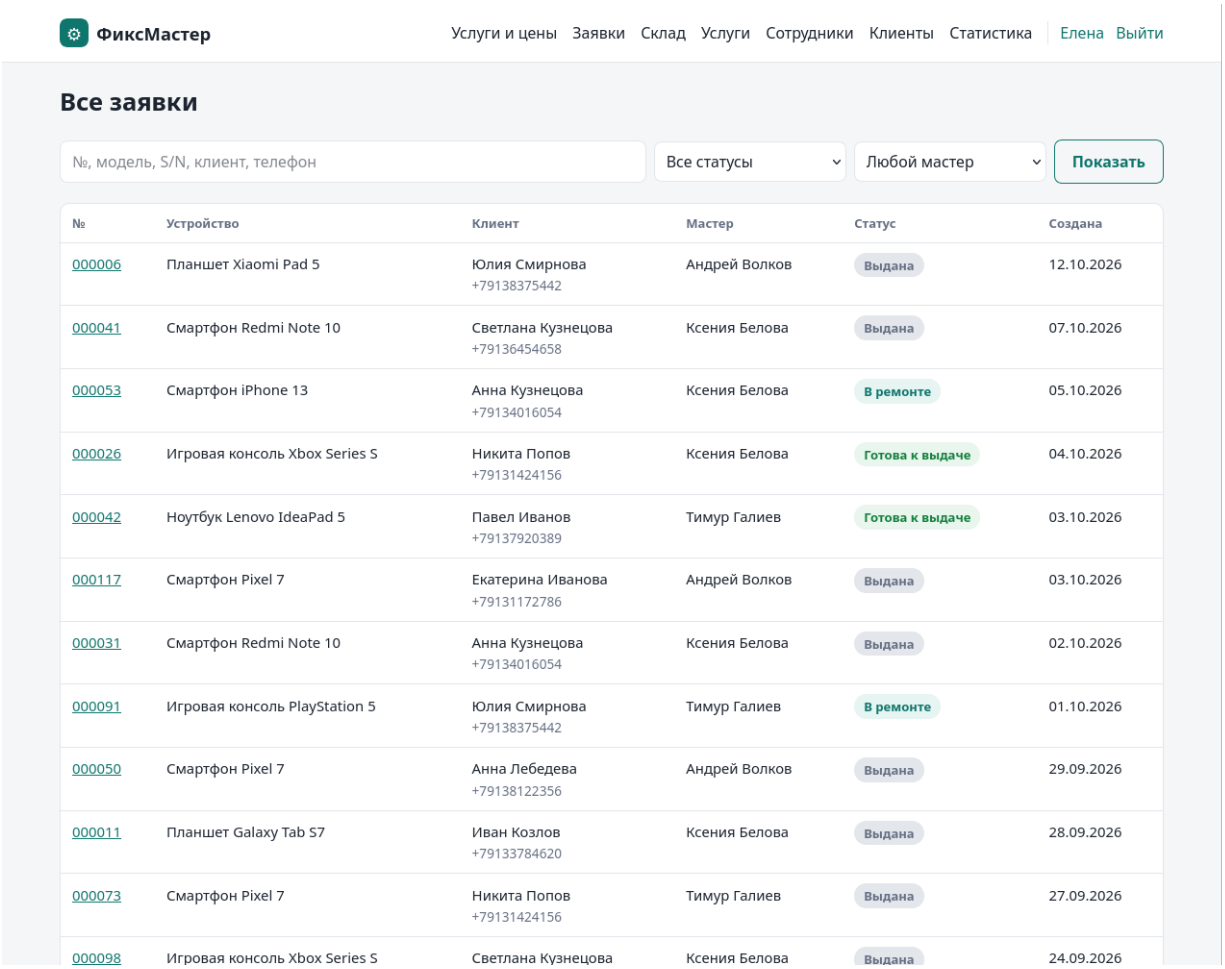


Рисунок 9 – Список заявок в панели администратора

3.9 Статистика

Страница статистики (рисунок 10) показывает ключевые показатели: заявки в работе, ожидающие выдачи, без мастера, доход за месяц, средний срок ремонта. Графики: заявки и доход по месяцам за год, популярные неисправности, загрузка мастеров, распределение по устройствам и статусам. Доход считается по выданным заявкам как сумма стоимости работ и запчастей. Данные доступны также в формате JSON.

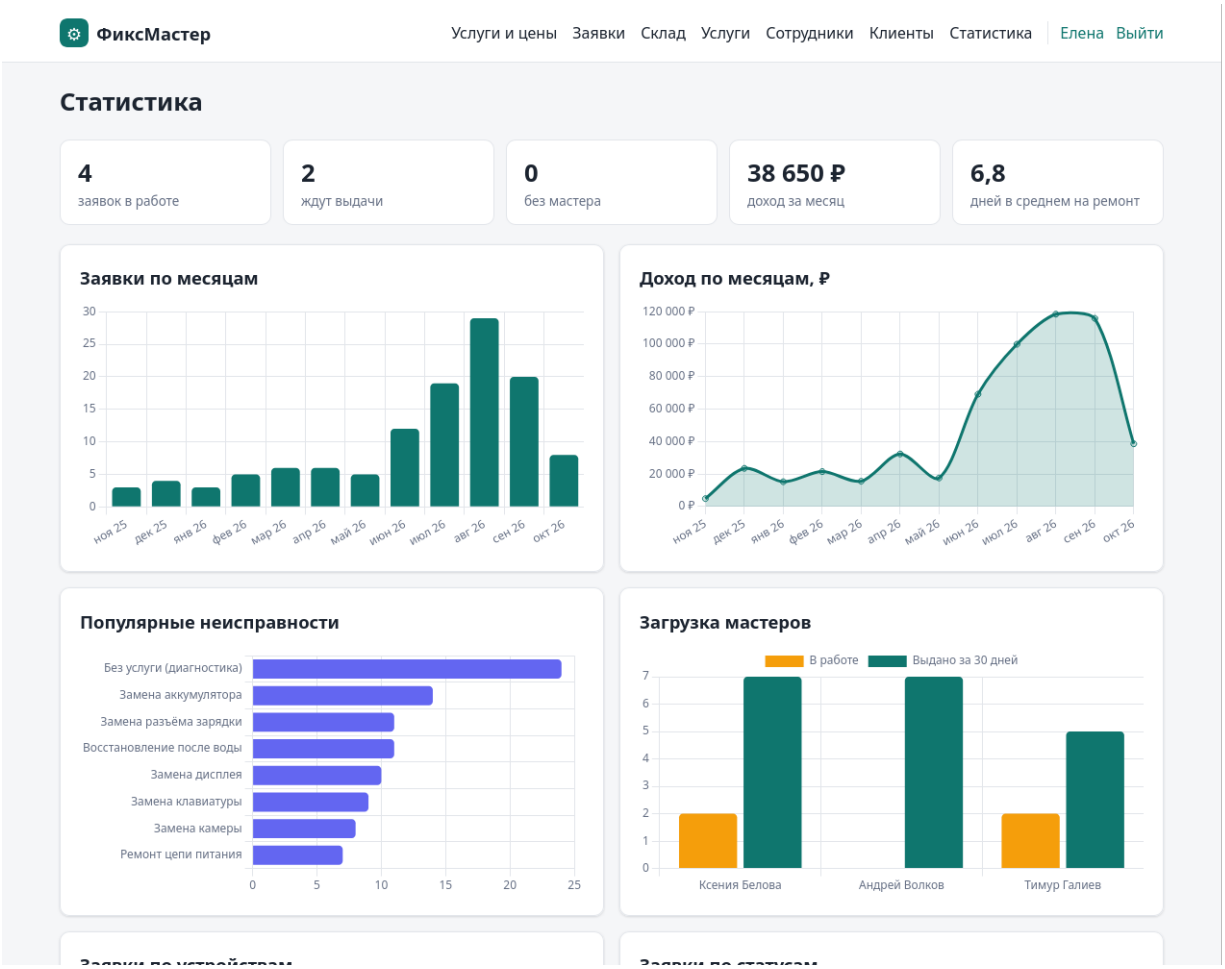


Рисунок 10 – Страница статистики

3.10 Адаптивность и безопасность

Вёрстка выполнена без CSS-фреймворков. На узком экране меню сворачивается в кнопку, колонки встают друг под другом (рисунок 11). Поддерживается тёмная тема по настройке операционной системы.

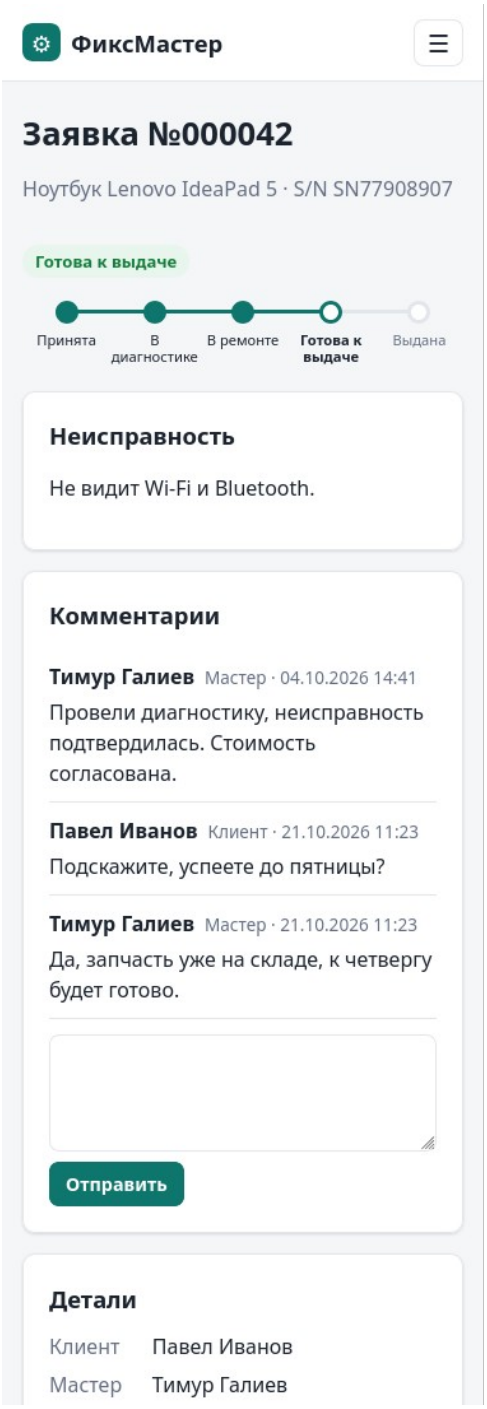


Рисунок 11 – Карточка заявки на экране смартфона

Меры безопасности:

– пароли хранятся только в виде хеша PBKDF2-SHA256 с солью, минимальная длина 8 символов, запрещены распространённые и чисто цифровые пароли;
– запросы к базе выполняются только через ORM с параметрами, сырого SQL в проекте нет;

– все формы защищены CSRF-токеном, изменение данных выполняется только методом POST;
– шаблоны автоматически экранируют вывод, что защищает от XSS;
– фотографии не раздаются напрямую, а отдаются через обработчик с проверкой прав;
– на сервере cookie передаются только по HTTPS, включены заголовки X-Frame-Options и X-Content-Type-Options.

4 Тестирование

4.1 Автоматические тесты

Для модулей написаны автоматические тесты на встроенном фреймворке django.test – всего 76 тестов (таблица 7). Тесты запускаются командой `python manage.py test` и проходят на обеих СУБД: SQLite и PostgreSQL 17.

Таблица 7 – Состав автоматических тестов

Модуль	Что проверяется
Учётные записи	Регистрация, хеширование пароля, невозможность зарегистрироваться администратором, уникальность e-mail, слабые пароли, границы телефона, обязательные поля, перенаправление по роли, вход заблокированного пользователя
Каталог	Группировка, поиск, фильтр, некорректные параметры, пустой результат
Заявки	Создание, фото (0, 2, 5, 6 файлов, большой файл, файл не-изображение), границы длины описания, доступ по ролям, защита фото, внутренние комментарии, публичная страница статуса
Процесс ремонта	Полный путь статусов, запрет пропуска этапов, права администратора, выдача без мастера, чужой мастер, списание и возврат запчастей, нехватка на складе, фиксация цены, блокировка выданной заявки
Склад	Приход, инвентаризация, признак «мало», права ролей, уникальность артикула, отрицательная цена
Панель администратора	Доступ только администратору, сотрудники, защита от самоблокировки, услуги, фильтры, поиск со спецсимволами SQL, расчёт статистики

4.2 Проверка форм и граничных значений

Таблица 8 – Проверка граничных значений

Поле	Значения	Ожидаемый результат
Описание неисправности	9 / 10 / 2000 / 2001 символ, только пробелы	ошибка / принято / принято / ошибка / ошибка
Количество фото	5 / 6 файлов	принято / ошибка
Файл фото	больше лимита; не изображение	ошибка
Телефон	9 цифр / 10 / «+» и 15 / 16 / с буквами	ошибка / принято / принято / ошибка / ошибка
Пароль	7 символов; только цифры; не совпадают	ошибка

Поле	Значения	Ожидаемый результат
Стоимость работ	−1 / abc / 0 / 1 000 000 / 1 000 000,01	ошибка / ошибка / принято / принято / ошибка
Цена услуги, срок	0 ₽ / −0,01 ₽ / 0 дней	принято / ошибка / ошибка
Списание запчасти	весь остаток / на 1 больше / 0	принято / ошибка / ошибка
Строка поиска	' OR 1=1 --; "; DROP TABLE …; %; _	страница работает, данные не повреждены

Все проверки дали ожидаемый результат.

4.3 Время отклика

Замер выполнен для приложения в Docker (Gunicorn, PostgreSQL 17) на демонстрационных данных: 120 заявок, 25 клиентов, 14 запчастей, по 20 запросов на страницу (таблица 9).

Таблица 9 – Время отклика страниц

Страница	Медиана, мс	Максимум, мс
Каталог услуг	13	38
Все заявки (администратор)	21	27
Статистика	26	32
Журнал склада	25	39
Карточка заявки	24	35

Требование «время отклика не более 2–3 секунд» выполняется с большим запасом. На опубликованной версии время ответа самого сервера составило 2–18 мс; при доступе из внешней сети основное время занимает установка соединения.

4.4 Ручное тестирование и найденные дефекты

Вручную пройдены сквозные сценарии: гость ищет услугу в каталоге; клиент регистрируется и создаёт заявку с фото; администратор назначает мастера; мастер проходит статусы, списывает запчасть и указывает стоимость; клиент видит прогресс, стоимость и публичные комментарии; остаток на складе уменьшается, в журнале появляется запись; статистика совпадает с

данными; на ширине 390 px нет горизонтальной прокрутки. В ходе тестирования найдены и исправлены дефекты, перечисленные в таблице 10.

Таблица 10 – Найденные и исправленные дефекты

№	Дефект	Исправление
1	Отправка пустой формы показывала форму без сообщений об ошибках	Форма связывается с данными по методу запроса
2	На экране смартфона карточка заявки выходила за край экрана	Ограничение минимальной ширины колонок CSS-сетки
3	У администратора в списке статусов по умолчанию стоял «Принята» вместо текущего	Начальное значение – текущий статус
4	Порядок клиентов на страницах списка мог меняться между запросами	Явная сортировка при агрегации

5 Развёртывание и публикация

Приложение упаковано в Docker-образ и запускается вместе с PostgreSQL через Docker Compose. Проект опубликован на сервере под управлением Debian 13 (рисунок 12). Внешние запросы принимает Apache с сертификатом Let's Encrypt и передаёт их приложению, которое слушает только локальный адрес. Порт базы данных наружу не открыт. HTTP-запросы перенаправляются на HTTPS.

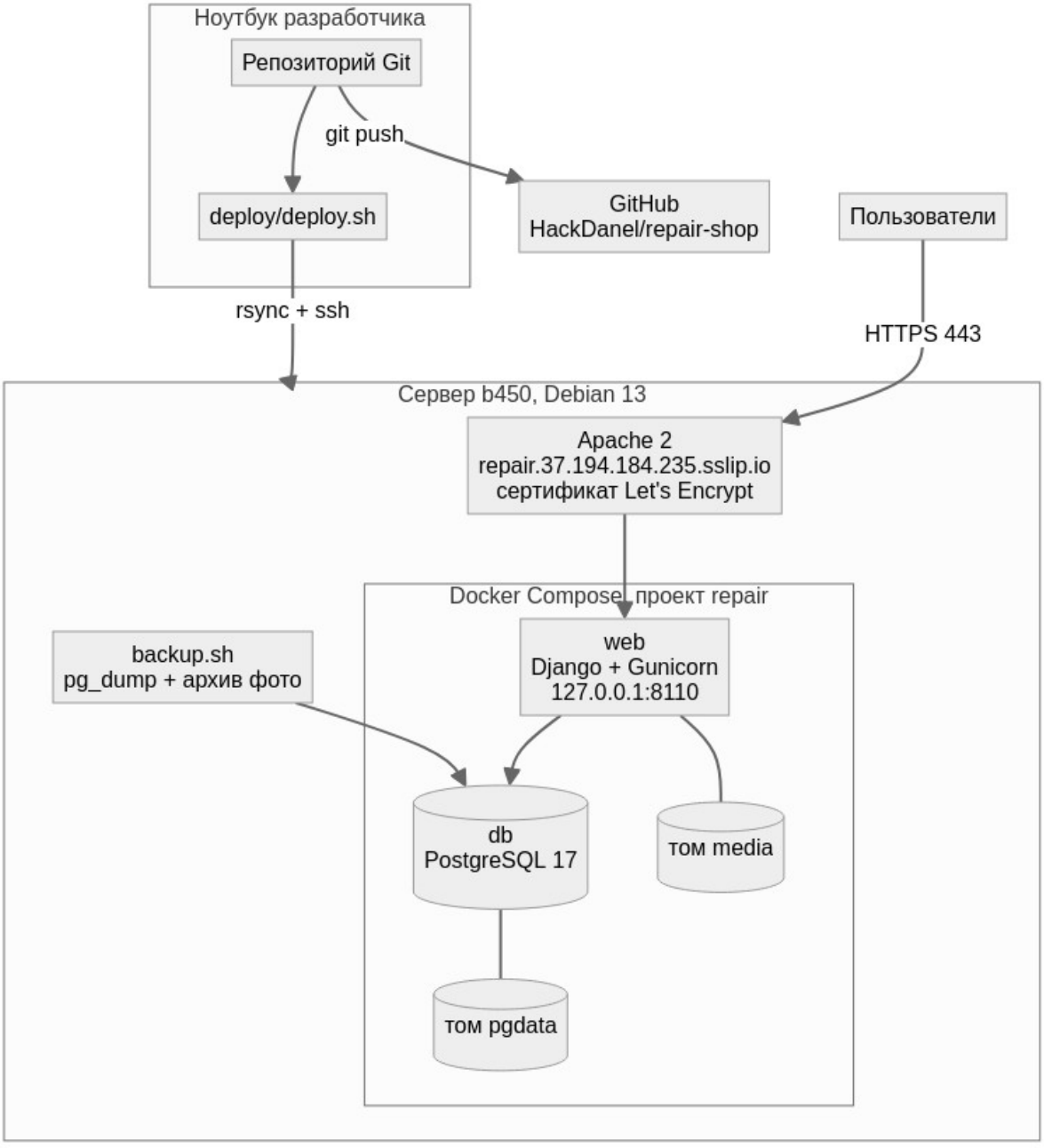


Рисунок 12 – Схема развёртывания

Адрес опубликованного приложения: https://repair.37.194.184.235.sslip.io.

Исходный код размещён в репозитории GitHub: https://github.com/HackDanel/repair-shop.

Обновление выполняется скриптом deploy/deploy.sh: он копирует код на сервер по SSH, пересобирает образ и перезапускает контейнер; миграции базы применяются автоматически. Скрипт backup.sh создаёт дамп базы данных и архив фотографий и хранит 14 последних копий. Порядок установки, обновления, резервного копирования и восстановления описан в технической документации (раздел 8 каталога docs).

Документация проекта в репозитории включает: анализ предметной области, требования, архитектуру, описание базы данных, отчёт о тестировании, руководство пользователя, техническое описание программы и руководство по развёртыванию.

ЗАКЛЮЧЕНИЕ

В ходе практики разработано и опубликовано веб-приложение «Ремонтная мастерская». Выполнены все поставленные задачи:

– проанализирована предметная область и существующие решения, сформулированы функциональные и нефункциональные требования;
– спроектированы архитектура приложения, схема статусов заявки и база данных из 11 таблиц с ограничениями целостности;
– реализованы каталог услуг, регистрация и авторизация с тремя ролями, создание заявки с фото, личный кабинет клиента, рабочее место мастера, учёт запчастей, панель администратора и страница статистики с графиками;
– написаны 76 автоматических тестов, проверены формы и граничные значения, время отклика страниц не превышает 40 мс на сервере;
– подготовлены техническая документация и руководство пользователя, проект опубликован с HTTPS и резервным копированием.

Из дополнительных возможностей реализованы графики статистики и публичная проверка статуса по коду, на которую будет вести QR-код квитанции. Направления развития: генерация акта приёма-передачи и квитанции в PDF с QR-кодом, уведомления клиентов по e-mail при смене статуса, Kanban-доска заявок для администратора.

В результате практики закреплены навыки анализа требований, проектирования баз данных, разработки веб-приложений на Python и Django, автоматического тестирования, работы с Git и развёртывания приложений в Docker.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1 ГОСТ 7.32-2017. Система стандартов по информации, библиотечному и издательскому делу. Отчёт о научно-исследовательской работе. Структура и правила оформления. – Москва : Стандартинформ, 2017. – 32 с.

2 Django documentation. Release 6.0 [Электронный ресурс]. – URL: https://docs.djangoproject.com/en/6.0/ (дата обращения: 21.10.2026).

3 Python 3.13 documentation [Электронный ресурс]. – URL: https://docs.python.org/3.13/ (дата обращения: 21.10.2026).

4 PostgreSQL 17 Documentation [Электронный ресурс]. – URL: https://www.postgresql.org/docs/17/ (дата обращения: 21.10.2026).

5 Chart.js documentation [Электронный ресурс]. – URL: https://www.chartjs.org/docs/latest/ (дата обращения: 21.10.2026).

6 Docker Docs. Docker Compose [Электронный ресурс]. – URL: https://docs.docker.com/compose/ (дата обращения: 21.10.2026).

7 Gunicorn documentation [Электронный ресурс]. – URL: https://docs.gunicorn.org/ (дата обращения: 21.10.2026).

8 Apache HTTP Server Version 2.4. mod_proxy [Электронный ресурс]. – URL: https://httpd.apache.org/docs/2.4/mod/mod_proxy.html (дата обращения: 21.10.2026).

9 OWASP Top 10 [Электронный ресурс]. – URL: https://owasp.org/www-project-top-ten/ (дата обращения: 21.10.2026).

10 MDN Web Docs. CSS Grid Layout [Электронный ресурс]. – URL: https://developer.mozilla.org/ru/docs/Web/CSS/CSS_grid_layout (дата обращения: 21.10.2026).

11 Pro Git / S. Chacon, B. Straub [Электронный ресурс]. – URL: https://git-scm.com/book/ru/v2 (дата обращения: 21.10.2026).

ПРИЛОЖЕНИЕ А

(обязательное)

Фрагменты исходного кода

Листинг А.1 – Списание запчасти со склада (apps/repairs/services.py)

```
def use_part(req, part_id, quantity, user):
    """Списать запчасть со склада в ремонт."""
    if not can_work_on(user, req):
        raise PermissionDenied
    if req.is_closed:
        raise ValidationError('Заявка уже выдана, менять состав работ
нельзя.')
    if quantity < 1:
        raise ValidationError('Количество должно быть не меньше 1.')
    with transaction.atomic():
        # Блокировка строки защищает от гонки, когда два мастера берут
последнюю деталь.
        part = Part.objects.select_for_update().get(pk=part_id)
        if part.quantity < quantity:
            raise ValidationError(f'На складе только {part.quantity} шт.
«{part.name}».')
        part.quantity -= quantity
        part.save(update_fields=['quantity'])
        StockMovement.objects.create(
            part=part, kind=StockMovement.Kind.USAGE, delta=-quantity,
request=req, author=user,
        )
        return PartUsage.objects.create(request=req, part=part,
quantity=quantity, unit_price=part.price)
```

Листинг А.2 – Смена статуса заявки (apps/repairs/services.py)

```
def change_status(req, new_status, user):
    if not can_work_on(user, req):
        raise PermissionDenied
    new_status = Status(new_status)
    # Администратор может исправить любой статус, мастер — только по схеме
переходов.
    if not user.is_admin_role and new_status not in req.allowed_statuses():
        raise ValidationError(f'Нельзя перевести заявку из
«{req.get_status_display()}» в «{new_status.label}».')
    if new_status == req.status:
        return req
    if new_status == Status.ISSUED and not req.master_id:
        raise ValidationError('Нельзя выдать заявку без назначенного
мастера.')
    with transaction.atomic():
        StatusChange.objects.create(request=req, old_status=req.status,
new_status=new_status, author=user)
        req.status = new_status
        req.issued_at = timezone.now() if new_status == Status.ISSUED else
None
        req.save(update_fields=['status', 'issued_at', 'updated_at'])
    return req
```

Листинг А.3 – Модель заявки и схема переходов статусов (apps/repairs/models.py, фрагмент)

```
class Status(models.TextChoices):
    ACCEPTED = 'accepted', 'Принята'
    DIAGNOSTICS = 'diagnostics', 'В диагностике'
    IN_REPAIR = 'in_repair', 'В ремонте'
    READY = 'ready', 'Готова к выдаче'
    ISSUED = 'issued', 'Выдана'


# Допустимые переходы статусов. Шаг назад разрешён, чтобы исправить ошибку
# (например, после диагностики понадобилась повторная), «Выдана» — конечный.
TRANSITIONS = {
    Status.ACCEPTED: [Status.DIAGNOSTICS],
    Status.DIAGNOSTICS: [Status.ACCEPTED, Status.IN_REPAIR, Status.READY],
    Status.IN_REPAIR: [Status.DIAGNOSTICS, Status.READY],
    Status.READY: [Status.IN_REPAIR, Status.ISSUED],
    Status.ISSUED: [],
}
```

ПРИЛОЖЕНИЕ Б

(справочное)

Учётные записи и ссылки

Таблица Б.1 – Ссылки на результаты

Ресурс	Адрес
Опубликованное приложение	https://repair.37.194.184.235.sslip.io
Репозиторий исходного кода	https://github.com/HackDanel/repair-shop
Документация	каталог docs/ репозитория

Таблица Б.2 – Демонстрационные учётные записи

Логин	Роль
client	Клиент
master1, master2, master3	Мастер
admin	Администратор

Пароль демонстрационных учётных записей предоставляется руководителю практики отдельно.